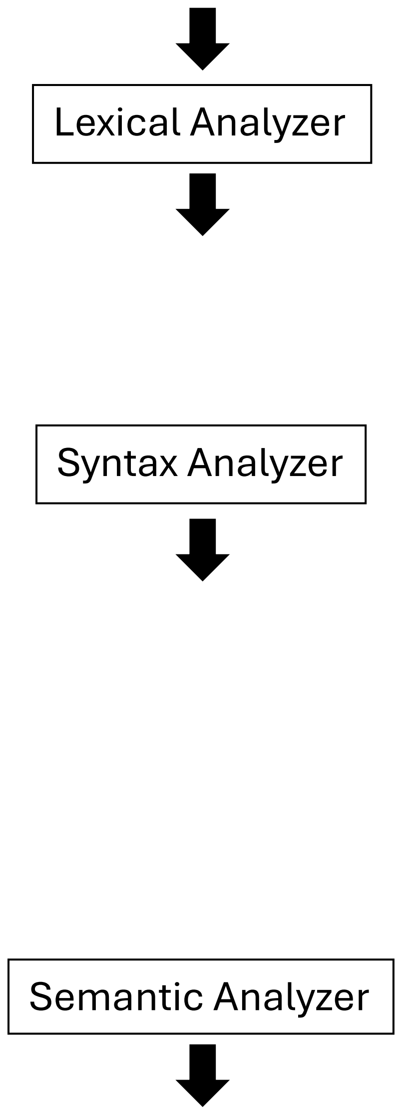
Q. No. 1: **What will be the output of the first three phases of compiler when the following assignment statement passes through them.**

area = length + 2.5 * (width * height) (All variables are of type 'real')

Solution:



Q. No. 2: MCQs.

1) What of the following is **NOT TRUE** about Lexical Analysis?

a) Return the next token to the scanner.

b) Ignores the blanks and comments.

c) Groups the characters into lexemes

d) Produce token stream.

2) Processes the source code only **once** and produces object code.

a) one pass compilers.

b) multi pass compilers.

c) Incremental Compilers.

d) Cross Compilers.

3) What of the following is **NOT TRUE** about Symbol Table?

a) Allow all front-end and back-end phases to find the records.

b) The lexical analyzer inserts identifiers into the symbol table when detected.

c) Contains information about the type, scope, or storage of identifiers.

d) Each compiler phase creates its own separate symbol table unrelated to others.

4) The **assembler** converts the assembly code into…

a) Intermediate code.

b) Relocatable machine code.

c) High-level source code.

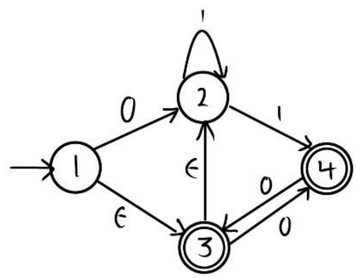
d) Absolute machine code.

5) How many tokens will be created by the lexical analyzer for the following statement?

While (10 > 1 && 7 = 5)

a) 8
b) 10
c) 11
d) 12

Q. No. 3: ***Convert the following NFA into its equivalent DFA, by showing all steps in detail. Without detailed calculations, you will get 0 marks.***



a) Calculations:

b) State Table for DFA:

	0	**1**

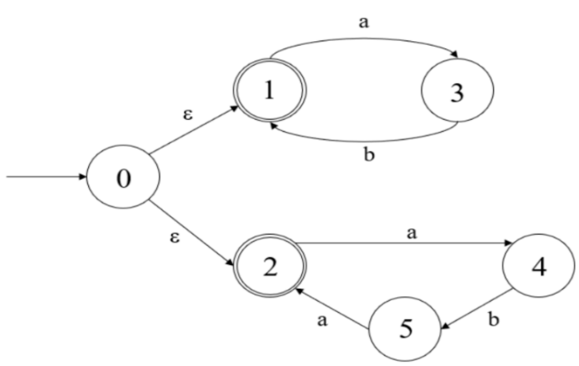
Q. No. 4: Solve these NFA questions:

a) Draw the NFA for the following regular expression: (a|b)* ab(a|b)

(The doctor said at least write 4 steps, not directly the final solution)

b) Check if the strings are accepted in the NFA below and draw their path:

(I couldn't remember the exact NFA but the idea is the same)



1- The pattern abaa is (Accepted/Rejected) using the above NFA.
   Draw the path:

2- The pattern abab is (Accepted/Rejected) using the above NFA.
   Draw the path:

Q. No. 5: **Consider the following grammar.**

$$E \rightarrow E \vee T \mid E \wedge T \mid T$$
$$T \rightarrow \neg T \mid D$$
$$D \rightarrow (E) \mid id$$

a) Construct a parse tree for the expression $\neg(id \wedge id) \vee id$

b) Is this grammar ambiguous? explain why.

Q. No. 6: **Consider the following grammar.**

$S \longrightarrow \text{if } B\, T\, ;$

$B \longrightarrow \text{true} \mid \text{false}$

$T \longrightarrow id = val$

$id \longrightarrow a \mid b$

$val \longrightarrow 0 \mid 1 \mid 2$

Use **right** derivation to produce this output: **if false a = 2;**

Q. No. 7: **Given this sample code:**

```
int total = 0 ;
While ( 100 > total ) {
        total = total + 10 ;
}
```

Fill this table:

Lexemes	Token_names	Does it enter the symbol table (Yes/No)